Project

Geavanceerde computerarchitectuur

Joeri Sevens & Arthur Valentin

December 15, 2024



Year:	2024
Class:	M ELEC
Instructor:	Thomas Feys

Contents

1 Inleiding **2**

2 Python **3**

3 C++ **4**

4 Cuda **4**

5 Metingen **4**

6 Uitbreidingen **5**

7 Besluit **6**

8 Code **7**
8.1 Python 7
8.2 C++ 13
8.3 Cuda V1 19
8.4 Cuda V2 28
8.5 Cuda V3 30
8.6 Cuda V4 32
8.7 Cuda V5 34
8.8 Cuda V6 36
8.9 CSV vergelijker 38
8.10 CSV visualiseren 39

1 Inleiding

Voor het project Geavanceerde Computerarchitectuur hebben we een programma ontwikkeld dat de verspreiding van elektromagnetische (EM) golven van een Wi-Fi-router simuleert. De simulatie berekent stap voor stap hoe deze golven zich door een tweedimensionale ruimte verspreiden, uitgaande van een punt dat de locatie van de router representeert.

We hebben voor dit project gekozen omdat we eerder een Python-implementatie van dergelijke simulaties hadden gemaakt tijdens een labo over EM-golven. Daarbij merkten we dat deze implementatie, zelfs in relatief kleine ruimtes, traag presteerde. De code bleek echter goed geschikt voor parallelle verwerking, wat ons motiveerde om te onderzoeken hoe de prestaties verbeterd kunnen worden door gebruik te maken van parallelle architecturen.

De simulatie wordt achtereenvolgens geïmplementeerd in Python, C++, en CUDA. Door de uitvoeringssnelheid van deze implementaties met elkaar te vergelijken, trekken we conclusies over de efficiëntie van de verschillende methodes. CUDA is bijzonder geschikt voor dit project omdat de simulatie grotendeels bestaat uit rekenintensieve, elementgewijze operaties op matrices, die eenvoudig parallel uitgevoerd kunnen worden.

De berekening van de elektrische (E) en magnetische (H) veldcomponenten volgt expliciete formules, die afgeleid kunnen worden uit de formules van Maxwell. Voor elke cel worden de waarden van E en H geüpdatet op basis van lokale verschillen met omliggende cellen in de andere matrix. Dit proces verloopt volgens onderstaande formules:

$$Ezx[iY, iX] = Cezx1[iY, iX] \cdot Ezx[iY, iX] + Cezx2[iY, iX] \cdot (Hy[iY, iX] - Hy[iY, iX - 1]) \quad (1)$$

$$Ezy[iY, iX] = Cezy1[iY, iX] \cdot Ezy[iY, iX] - Cezy2[iY, iX] \cdot (Hx[iY, iX] - Hx[iY - 1, iX]) \quad (2)$$

$$Ez = Ezx + Ezy \quad (3)$$

$$Hx[iY, iX] = Chx1[iY, iX] \cdot Hx[iY, iX] - Chx2[iY, iX] \cdot (Ez[iY + 1, iX] - Ez[iY, iX]) \quad (4)$$

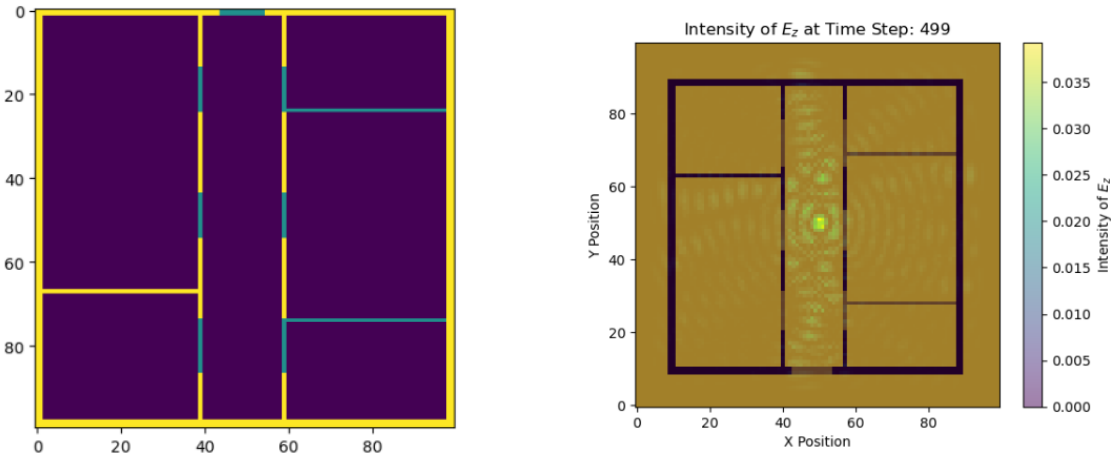
$$Hy[iY, iX] = Chy1[iY, iX] \cdot Hy[iY, iX] + Chy2[iY, iX] \cdot (Ez[iY, iX + 1] - Ez[iY, iX]) \quad (5)$$

Deze formules maken uitsluitend gebruik van waarden uit de cel zelf en naburige cellen van de andere matrix. Dankzij dit ontwerp treden er geen race conditions op, omdat altijd eerst de E-matrix en daarna de H-matrix wordt geüpdatet. Hierdoor is het mogelijk de berekeningen op een GPU te paralleliseren, waarbij elke thread verantwoordelijk is voor een specifiek deel van de matrix.

De gebruikte ruimte wordt weergegeven in Figuur 1a. Deze ruimte is ontworpen als een matrix met verschillende permeabiliteiten, waarbij muren en deuren hogere weerstand bieden tegen de voortplanting van EM-golven. Voor een eerlijke vergelijking tussen de programmeertalen exporteren we deze ruimte vanuit Python naar een CSV-bestand, zodat deze consistent blijft.

Indien de tijd het toelaat, willen we extra functionaliteit toevoegen waarmee specifieke punten op een plattegrond gemarkeerd kunnen worden waar optimale signaalsterkte gewenst is. Het algoritme zal vervolgens de ideale routerlocatie bepalen om op deze punten het sterkst mogelijke signaal te garanderen.

Tot slot visualiseren we de voortplanting van de golven met behulp van Python. Een voorbeeld van deze simulatie is te zien in Figuur 1b.



(a) Lege ruimte (b) Voorbeeld simulatie

Figure 1: De gesimuleerde ruimte (links) en een visualisatie van de golven (rechts).

2 Python

De Python-code hebben we oorspronkelijk ontwikkeld voor een labo van het vak *Elektromagnetische golven en antennes*. In deze code worden eerst enkele fysische constanten gedefinieerd, waarna een simulatieruimte wordt gecreëerd. Deze ruimte stelt een kamer voor, waarbij gebieden met een hogere permeabiliteit muren of deuren representeren. Om ervoor te zorgen dat de waarden in de simulatie niet overal nul blijven, wordt er in het midden van de kamer een bron (*source*) toegevoegd.

Om een eerlijke vergelijking tussen de verschillende implementaties te garanderen, hebben we de ruimte vanuit de Python-code geëxporteerd naar een CSV-bestand. Dit bestand kan vervolgens worden ingelezen en gebruikt in de andere implementaties.

Na het definiëren van de ruimte en enkele variabelen, komt de kern van de Python-code: de berekening van het elektromagnetische veld. In een **for-loop** worden eerst E_{zx} en E_{zy} berekend, waarna deze worden samengevoegd tot E_z. Vervolgens worden H_x en H_y berekend. Deze berekeningen worden periodiek uitgevoerd, waarna de resultaten worden weergegeven.

De ruimte en een voorbeeld van de simulatie worden hieronder weergegeven in Figuur 1.

Bij het meten van de benodigde tijd voor één frame hebben we uitsluitend rekening gehouden met de berekening van de E- en H-waarden, zonder de tijd voor visualisatie mee te nemen, zoals ook in de code zichtbaar is.

Omdat visualisaties in verschillende talen altijd enigszins anders zouden zijn, leek het ons niet zinvol om de visualisatie in elke taal opnieuw uit te werken. Dit zou immers een eerlijke vergelijking bemoeilijken. Daarom hebben we een aanvullende Python-code geschreven om de E_z-waarden, opgeslagen in CSV-bestanden vanuit de andere implementaties, te vergelijken. Daarnaast hebben we een Python-script ontwikkeld dat deze waarden op dezelfde manier visualiseert als in de oorspronkelijke code.

3 C++

Na de implementatie in Python zijn we begonnen met het omzetten van de code naar C++. Hoewel C++ net als Python een sequentiële programmeertaal is die op de CPU draait, is C++ aanzienlijk sneller dankzij de manier waarop de taal is opgebouwd. De C++-code werkt op dezelfde manier als de Python-implementatie, maar bevat twee belangrijke verschillen.
Ten eerste creëren we de simulatieruimte niet opnieuw, maar lezen we deze in vanuit een CSV-bestand dat eerder in Python is gegenereerd. Dit zorgt voor consistentie tussen de implementaties en maakt een eerlijke vergelijking van de uitvoeringstijden mogelijk.
Ten tweede hebben we de visualisatie van de E_z-matrices verwijderd in de C++-code. In plaats daarvan schrijven we deze waarden weg naar een CSV-bestand. Dit bestand kan vervolgens worden ingelezen door de eerder beschreven Python-code, die dezelfde visualisatiemethode gebruikt als in de oorspronkelijke Python-implementatie.

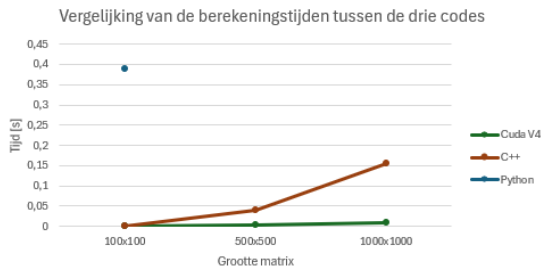
4 Cuda

De derde taal die we hebben geïmplementeerd is CUDA. Dit is de eerste taal die de berekeningen parallel uitvoert, aangezien deze draait op de GPU. Het voordeel hiervan is dat verschillende elementen van de matrices tegelijkertijd berekend kunnen worden, wat aanzienlijke tijdswinst oplevert. Omdat CUDA het belangrijkste doel van deze opdracht was, hebben we verschillende versies ontwikkeld om de prestaties geleidelijk te verbeteren.
De berekeningen van de frames zijn in CUDA letterlijk overgenomen uit de C++-code. De enige aanpassingen bevinden zich binnen de for-loop die elk frame berekent en in de kernels waarnaar in deze loop wordt verwezen. Daarom hebben we enkel versie 1 volledig toegevoegd in de code achteraan, en bij de latere versies hebben we alleen de gewijzigde kernels opgenomen om de code overzichtelijker te maken.
Versie 1 houdt in dat alle elementen van de matrices in plaats van in een dubbele for-loop op de CPU, nu in een kernel op de GPU worden berekend. Versies 2 tot en met 4 proberen de code verder te optimaliseren door zoveel mogelijk kernels samen te voegen. Dit zou in theorie tijd moeten besparen, omdat hierdoor minder keren tussen de CPU en de GPU gewisseld hoeft te worden.
In versie 5 en 6 hebben we shared memory toegevoegd aan de kernels, in de veronderstelling dat dit de prestaties zou verbeteren, omdat meerdere elementen hetzelfde geheugen delen. Uit de latere metingen bleek echter dat deze aanpassing de prestaties niet bevorderde.

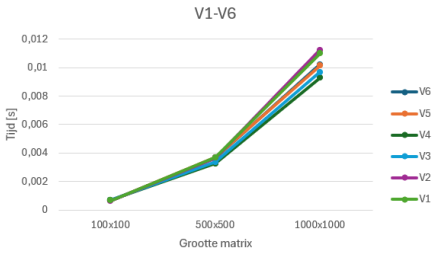
5 Metingen

Nadat we alle codes hadden geïmplementeerd, wilden we nagaan of de opeenvolgende versies daadwerkelijk sneller waren. We hebben dit gedaan door de tijd te meten die nodig is om elke frame te berekenen en hier het gemiddelde van te nemen. Deze meting werd drie keer uitgevoerd voor elke code, en aangezien elke code 500 frames berekent, hebben we het gemiddelde genomen over 1500 metingen in totaal. We observeerden een aanzienlijk verschil in berekeningstijd tussen de Python- en C++-codes, en wederom een groot verschil met de CUDA-code. Binnen de CUDA-versies zelf waren de verschillen in berekeningstijd echter zeer klein, wat naar onze mening binnen de foutmarges van de tijdsmetingen viel. Daarom besloten we om de metingen uit te voeren voor grotere ruimtes. Hiervoor creëerden we een ruimte van 500x500 en 1000x1000. In dit geval zagen we veel duidelijker de verschillen tussen de verschillende implementaties.

In de afbeelding van de gemiddelde berekeningstijd hebben we echter alleen de tijd van de Python-code voor een 100x100 matrix weergegeven, zie figuur 2. Dit is omdat de berekeningstijden voor grotere matrices (500x500 en 1000x1000) bij Python zodanig groot werden, dat het verschil in tijd tussen de C++- en CUDA-code, die relatief groot is, niet meer goed zichtbaar was. Door deze beperking konden we het verschil in prestaties tussen de verschillende implementaties duidelijker visualiseren voor de kleinere matrix.



(a) Gemiddelde berekeningstijd



(b) Vergelijking gemiddelde berekeningstijd CUDA

Figure 2: Vergelijking van de gemiddelde berekeningstijden tussen Python en de CUDA-versies

Table 1: Gemiddelde berekeningstijden van python, c++ en cuda

Grootte Matrix	**Python**	**C++**	**Cuda V4**
100x100	0,38826	0,001692497	0,000680582
500x500	1,507	0,0393278	0,0032638
1000x1000	5,07636	0,155141667	0,009273243

Table 2: Gemiddelde berekeningstijden voor verschillende matrixgroottes

Grootte Matrix	**V1**	**V2**	**V3**	**V4**	**V5**	**V6**
100x100	0,00066187	0,00063829	0,00069789	0,000680582	0,000655503	0,000671968
500x500	0,00368392	0,00365989	0,00337027	0,0032638	0,00367965	0,003656687
1000x1000	0,01099247	0,0112358	0,00966566	0,009273243	0,0101367	0,010205833

6 Uitbreidingen

In de huidige implementatie berekenen we uitsluitend de Z-component van het elektrische veld. Dit is correct, aangezien we slechts een bron hebben toegevoegd die golven in de Z-richting veroorzaakt. In de werkelijkheid zal een bron echter vrijwel altijd in alle richtingen uitstralen, wat betekent dat er ook een X- en Y-component van het elektrische veld aanwezig zullen zijn. Dit zou het aantal berekeningen per frame verdubbelen.

Daarnaast is het mogelijk om de formules uit te breiden zodat we met een 3D-matrix kunnen werken, waarmee we de golven in een driedimensionale ruimte zouden kunnen berekenen.

Verder zouden we, op basis van de huidige code, een extra lus kunnen toevoegen die de bron op verschillende locaties plaatst. Dit zou ons in staat stellen om de optimale locatie voor een wifi-punt te bepalen door de beste wifi-ontvangst op vooraf gedefinieerde locaties te meten.

Al deze uitbreidingen zouden echter de complexiteit van de berekeningen vergroten en zouden aanzienlijke rekenkracht vereisen. Elke milliseconde die we winnen in het berekenen van een frame of een component van een frame heeft een aanzienlijke invloed op de totale rekentijd, vooral gezien het feit dat de totale rekentijd afhangt van een groot aantal frames en een veel grotere ruimte dan de huidige implementatie.

7 Besluit

Python was naar verwachting de eenvoudigste taal om in te programmeren en de code te visualiseren, vooral binnen de Jupyter Notebook-omgeving. Echter, zoals de cijfers aantonen in tabel 1, blijkt Python in vergelijking met de andere talen extreem traag te zijn. In C++ zagen we al een aanzienlijke verbetering in snelheid, maar de uitvoering bleef nog niet optimaal, vooral bij grotere matrices, omdat de code nog steeds sequentieel werd uitgevoerd.

Met CUDA verwachtten we de snelste berekeningstijd, en dit bleek inderdaad het geval te zijn, zoals te zien is in Figuur 2. We hadden gehoopt dat het gebruik van shared memory de berekeningstijden, vooral voor grotere matrices, verder zou versnellen. Het gebruik van shared memory resulteerde echter in een vertraging, waarbij de snelste tijd werd behaald door versie V4, zoals weergegeven in Figuur 2b en tabel 2. De verkorte berekeningstijd door het verminderen van het aantal kernels werd tenietgedaan door de extra overhead van het kopiëren van gegevens naar het shared memory, wat de prestaties negatief beïnvloedde.

Concluderend kunnen we zeggen dat de CUDA-code versie V4, met samengevoegde kernels, de beste prestaties levert. We verwachten dat dit prestatieverschil ten opzichte van Python en C++ nog groter zal zijn wanneer we in de toekomst met 3D-matrices werken, bijvoorbeeld voor het visualiseren van wifi in een huis.

Alle implementaties werkten zoals verwacht, wat we bevestigden door de Python-code te gebruiken om de CSV-bestanden met elkaar te vergelijken. Daarnaast konden we de werking verder visualiseren met een ander Python-script. Door tijdgebrek konden we de code echter niet uitbreiden naar een versie die in staat zou zijn om het optimale punt voor maximale dekking op zelfgekozen locaties te berekenen. Dit zou een interessante uitbreiding kunnen zijn voor de toekomst.

8 Code

8.1 Python

```
import math
import matplotlib.pyplot as plt
import numpy as np
from IPython import display


Nx = 100
Ny = 100                            #number of samples
dx = 0.02
dy = 0.02                               #spatial step 2cm

# ----------------------------------------
# -      physical constants              -
# ----------------------------------------
c    = 2.99792458e8                         #speed of light in vacuum
mu   = 4.0*math.pi*1.0e-7                       #permeability of free
    space
e_0  = 1.0/(c**2*mu)                         #permittivity of free space
    F/m
e_r  = np.ones((Nx, Ny))                #relative permittivity, USE THE
    RIGHT DATA TYPE!

# ----------------------------------------
# -      temporal information            -
# ----------------------------------------
dt = 1.0/(c*math.sqrt( (1/dx)**2 + (1/dy)**2 ))  #time step (2D CFL-
    condition)
Nt = 500                                    #number of time steps

# ----------------------------------------
# -      field variable allocation       -
# ----------------------------------------
Hx = np.zeros((Nx, Ny))                         #magnetic field x-
    component
Hy = np.zeros((Nx, Ny))                            #magnetic field y-
    component
Ez = np.zeros((Nx, Ny))                             #electric field z-
    component
Ezx = np.zeros((Nx, Ny))                            #electric field z-
    component in the x-direction
Ezy = np.zeros((Nx, Ny))                           #electric field z-
    component in the y-direction

# ----------------------------------------
# -          source definition           -
# ----------------------------------------
sigma_t = 10*dt                             #width of the pulse
t0      = 3*sigma_t                         #center of the pulse
wavel   = 0.15                              #wavelength [m]
```

```
41 N_lamb  = c/wavel                              #frequency [Hz]
42
43 # ------- gaussian pulse ------
44 # source  = @(t) -2*(t*dt-t0)./sigma_t .* exp(-((t*dt-t0)./sigma_t).^2);
45
46 # ------- sinusoid ------
47 source  = lambda t: math.sin(2*math.pi*N_lamb*(t-t0)*dt)
48
49 # ------- gaussian modulated sinusoid ------
50 #source  = @(t) sin(2*pi*N_lamb*(t-t0)*dt).*exp(-((t*dt-t0)./sigma_t).^2)
       ;
51
52 # ------- logistic modulated sinusoid ------
53 # source  = @(t) 0.5*(tanh((t-60)/15)+1).*sin(2*pi*N_lamb*(t-t0)*dt);
54
55 def makeRoom(room, wall_perm, door_perm):
56
57     wt = 2
58     it = 2
59     dt = 6
60
61     room = room - 1
62     wall_perm = wall_perm - 1
63     door_perm = door_perm - 1
64
65     perm_ratio = door_perm / wall_perm
66
67     #perimeter
68     room[:it, :] = wall_perm
69     room[-it:, :] = wall_perm
70     room[:, :it] = wall_perm
71     room[:, -it:] = wall_perm
72
73     #hall
74     nx, ny = np.shape(room)
75     nx4_10 = round(4*nx/10)
76     nx6_10 = round(6*nx/10)
77
78     room[:, nx4_10-1:nx4_10+wt-2] = wall_perm
79     room[:, nx6_10-1:nx6_10+wt-2] = wall_perm
80
81     #left rooms
82     ny2_3 = round(2*ny/3);
83
84     room[ny2_3:ny2_3+wt-1, 1:nx4_10] = wall_perm
85
86     #right rooms
87     ny1_4 = round(1*ny/4)
88     ny3_4 = round(3*ny/4)
89
90     room[ny1_4-1:ny1_4+wt-2, nx6_10+wt-2:] = wall_perm
91     room[ny3_4-1:ny3_4+wt-2, nx6_10+wt-2:] = wall_perm
92
```

```
93      #attribute doors
94      nx1_2 = round(nx/2)
95      ny9_10 = round(9*ny/10)
96      ny1_10 = round(ny/10)
97
98      #to hal
99      room[:it, nx1_2-dt:nx1_2+dt-1] = room[:it, nx1_2-dt:nx1_2+dt-1]*
            perm_ratio
100
101     #to top rooms
102     room[ny1_10-dt+10:ny1_10+dt+9, wt:-wt] = room[ny1_10-dt+10:ny1_10+dt
            +10-1, wt:-wt]*perm_ratio
103
104     #to bottom rooms
105     room[ny9_10-dt-10:ny9_10+dt-9, wt:-wt] = room[ny9_10-dt-10:ny9_10+dt
            -9, wt:-wt]*perm_ratio
106
107     #to middle right room
108     room[nx1_2-dt:nx1_2+dt-1, nx4_10-1:-wt] = room[nx1_2-dt:nx1_2+dt-1,
            nx4_10-1:-wt]*perm_ratio
109
110     room = room + 1
111     return room
112
113
114 def setBackgroundVariables_I(Nx,Ny,e_r,e_0,mu,dt,dx,dy):
115     #----------------------------------------
116     #-          PML boundaries              -
117     #----------------------------------------
118     dLayers = 10                                    #number of layers
119     order   = 3                                              #order of
            polynomial of conductivity values
120     s_x = np.zeros((Ny,Nx))
121     s_y = np.zeros((Ny,Nx))       #electric conductivities
122
123     for iX in range(Nx):
124         s_x[iX, :dLayers] = np.power(list(range(dLayers-1,-1,-1)), order)
                / (dLayers-1)**order
125         s_x[iX,-dLayers:] = np.power(list(range(dLayers)), order) / (
                dLayers-1)**order
126
127     for iY in range(Ny):
128         s_y[:dLayers,iY]  = np.power(list(range(dLayers-1,-1,-1)), order)
                / (dLayers-1)**order
129         s_y[-dLayers:,iY] = np.power(list(range(dLayers)), order) / (
                dLayers-1)**order
130
131     ss_x = s_x * mu / e_0
132     ss_y = s_y * mu / e_0                       #magnetic "conductivities
            "
133     #----------------------------------------
134     #-         wall material props.         -
135     #----------------------------------------
```

```
136     wall_permittivity = 3.0
137     door_permittivity = 2.0
138 
139     e_r[dLayers-1:-dLayers,dLayers-1:-dLayers] = makeRoom(e_r[dLayers-1:-
            dLayers,dLayers-1:-dLayers], wall_permittivity, door_permittivity)
140 
141     e_r_temp = e_r-1
142     ind = np.nonzero(e_r)
143     for el in ind:
144         s_x[el[0], el[1]] = e_r[el[0], el[1]]/40
145         s_y[el[0], el[1]] = e_r[el[0], el[1]]/40
146 
147 
148     Cezx1 = np.divide((e_0*e_r-0.5*dt*s_x),(e_0*e_r+0.5*dt*s_x))      #Ezx
            first term coefficient
149     Cezx2 = dt/(dx*(e_0*e_r+0.5*dt*s_x))                              #Ezx
            second term coefficient
150     Cezy1 = np.divide((e_0*e_r-0.5*dt*s_y),(e_0*e_r+0.5*dt*s_y))      #Ezy
            first term coefficient
151     Cezy2 = dt/(dy*(e_0*e_r+0.5*dt*s_y))                              #Ezy
            second term coefficient
152 
153     Chx1  = np.divide((mu-0.5*dt*ss_x),(mu+0.5*dt*ss_x))              #Hx
           first term coefficient
154     Chx2  = dt/(dy*(mu+0.5*dt*ss_x))                                  #Hx
           second term coefficient
155 
156     Chy1  = np.divide((mu-0.5*dt*ss_y),(mu+0.5*dt*ss_y))              #Hy
           second term coefficient
157     Chy2  = dt/(dx*(mu+0.5*dt*ss_y))                                  #Hy
           second term coefficient
158     return [Cezx1,Cezx2,Cezy1,Cezy2,Chx1,Chx2,Chy1,Chy2,e_r]
159 
160 #This is how you make a room
161 room = np.ones((100,100))
162 room = makeRoom(room, wall_perm = 3, door_perm = 2)
163 fig = plt.imshow(room)
164 
165 #You don't have to do this yourself. This is done in the function call in
        the block below.+
166 
167 # ----------------------------------------
168 # -         helper variables             -
169 # ----------------------------------------
170 midX   = round(Nx/2)                                  # midpoint in the
        x-direction
171 midY   = round(Ny/2)                                  # midpoint in the
        y-direction
172 quartX = round(Nx/4)                                  # quarterpoint in
        the x-direction
173 quartY = round(Ny/4)                                  # quarterpoint in
        the y-direction
174 
```

```
Cezx1,Cezx2,Cezy1,Cezy2,Chx1,Chx2,Chy1,Chy2,e_r =
    setBackgroundVariables_I(Nx,Ny,e_r,e_0,mu,dt,dx,dy)

from matplotlib.colors import LinearSegmentedColormap

x_disturbance, y_disturbance = 50 , 50 #choose location for disturbance

cMap = []
for value, colour in zip([0, 1],["Yellow", "Black"]):
    cMap.append((value, colour))

#------ iterate from t=0 until t=Nt-1 (or iterate Nt times) ------
for t in range(Nt):

    #------ solve dtEzx = dxHy ------
    for iX in range(1, Nx):
        for iY in range(Ny):
            Ezx[iY,iX] = Cezx1[iY,iX]*Ezx[iY,iX ]+ Cezx2[iY,iX]*( (Hy[iY,
                iX]-Hy[iY,iX-1]) )

    #------ solve dtEzy = - dyHx ------
    for iX in range(Nx):
        for iY in range(1, Ny):
            Ezy[iY,iX] = Cezy1[iY, iX] * Ezy[iY, iX] - Cezy2[iY, iX] * (
                Hx[iY, iX] - Hx[iY - 1, iX])

    #------ recouple Ez ------
    Ez = Ezx+Ezy

    #------ add source to the electric field ------
    Ez[x_disturbance, y_disturbance] += source(t)

    #------ solve dtHx = -dyEz ------
    for iX in range(Nx):
        for iY in range(Ny-1):
            Hx[iY,iX] = Chx1[iY,iX]*Hx[iY,iX] - Chx2[iY,iX]*( Ez[iY+1,iX]
                 - Ez[iY,iX] )


    #------ solve dtHy = dxEz ------
    for iX in range(Nx-1):
        for iY in range(Ny):
            Hy[iY,iX] = Chy1[iY, iX] * Hy[iY, iX] + Chy2[iY, iX] * (Ez[iY
                , iX + 1] - Ez[iY, iX])


    intensity = np.abs(Ez) ** 2

    plt.imshow((e_r-1), cmap=LinearSegmentedColormap.from_list("custom",
        cMap), alpha=1)

    plt.imshow(intensity, cmap='viridis', interpolation='nearest', origin
        ='lower', alpha = 0.5)
```

```
221     plt.colorbar(label='Intensity␣of␣$E_z$')
222     plt.title(f'Intensity␣of␣$E_z$␣at␣Time␣Step:␣{t}')
223     plt.xlabel('X␣Position')
224     plt.ylabel('Y␣Position')
225     plt.pause(0.1)  # Pause for a brief moment to view the plot
226     display.clear_output(wait=True)
```

8.2 C++

```cpp
#include <iostream>
#include <cmath>
#include <vector>
#include <fstream>
#include <sstream>
#include <tuple>
#include <algorithm> // For std::transform
#include <chrono> // For timing

using namespace std;
using namespace std::chrono;

// Define constants with #define
#define NX 100
#define NY 100
#define DX 0.02
#define DY 0.02
#define C 3e8
#define MU (4.0 * M_PI * 1.0e-7)
#define E_0 (1.0 / (C * C * MU))
#define NT 500

// Function to create a 2D matrix of doubles with given dimensions and initial value
vector<vector<double>> create2DArray(int Nx, int Ny, double initial_value = 0.0) {
    return vector<vector<double>>(Nx, vector<double>(Ny, initial_value));
}

// Function to read a CSV file and load the data into a matrix
vector<vector<double>> readCSV(const string& filename) {
    vector<vector<double>> matrix;
    ifstream file(filename);

    if (!file.is_open()) {
        cerr << "Fout bij het openen van bestand: " << filename << endl;
        return matrix;
    }

    string line;
    while (getline(file, line)) {
        stringstream ss(line);
        string value;
        vector<double> row;

        while (getline(ss, value, ',')) {
            row.push_back(stod(value));
        }
        matrix.push_back(row);
    }

```

```
50      file.close();
51      return matrix;
52  }
53
54  // Function to write a matrix to a CSV file
55  void appendMatrixToCSV(ofstream &file, const vector<vector<double>>&
        matrix, int t) {
56      if (!file.is_open()) {
57          cerr << "Fout bij het openen van bestand voor append: " << endl;
58          return;
59      }
60
61      // Write a header or delimiter for each frame
62      file << "Frame " << t << endl;
63
64      for (const auto& row : matrix) {
65          for (size_t i = 0; i < row.size(); ++i) {
66              file << row[i];
67              if (i < row.size() - 1) {
68                  file << ",";
69              }
70          }
71          file << endl;
72      }
73
74      // Optionally, write a blank line to separate frames
75      file << endl;
76  }
77
78  // Function to set background variables
79  tuple<vector<vector<double>>, vector<vector<double>>, vector<vector<
        double>>, vector<vector<double>>, vector<vector<double>>, vector<
        vector<double>>, vector<vector<double>>, vector<vector<double>>,
        vector<vector<double>>> setBackgroundVariables_I(int Nx, int Ny,
        vector<vector<double>> e_r, double e_0, double mu, double dt, double
        dx, double dy) {
80      int dLayers = 10;  // Number of layers
81      int order = 3;  // Order of polynomial of conductivity values
82
83      vector<vector<double>> s_x = create2DArray(Ny, Nx);
84      vector<vector<double>> s_y = create2DArray(Ny, Nx);  // Electric
            conductivities
85
86      for (int iX = 0; iX < Nx; ++iX) {
87          for (int j = 0; j < dLayers; ++j) {
88              s_x[iX][j] = pow(dLayers - 1 - j, order) / pow(dLayers - 1,
                    order);
89              s_x[iX][Nx - dLayers + j] = pow(j, order) / pow(dLayers - 1,
                    order);
90          }
91      }
92
93      for (int iY = 0; iY < Ny; ++iY) {
```

```
94          for (int j = 0; j < dLayers; ++j) {
95              s_y[j][iY] = pow(dLayers - 1 - j, order) / pow(dLayers - 1,
                    order);
96              s_y[Ny - dLayers + j][iY] = pow(j, order) / pow(dLayers - 1,
                    order);
97          }
98      }
99
100     vector<vector<double>> ss_x = s_x;
101     vector<vector<double>> ss_y = s_y;
102
103     for (int i = 0; i < Ny; ++i) {
104         for (int j = 0; j < Nx; ++j) {
105             ss_x[i][j] *= mu / e_0;
106             ss_y[i][j] *= mu / e_0;
107         }
108     }
109
110     for (int i = dLayers - 1; i < Ny - dLayers; ++i) {
111         for (int j = dLayers - 1; j < Nx - dLayers; ++j) {
112             if (e_r[i][j] != 1) {
113                 s_x[i][j] = e_r[i][j] / 40.0;
114                 s_y[i][j] = e_r[i][j] / 40.0;
115             }
116         }
117     }
118
119     vector<vector<double>> Cezx1 = create2DArray(Ny, Nx);
120     vector<vector<double>> Cezx2 = create2DArray(Ny, Nx);
121     vector<vector<double>> Cezy1 = create2DArray(Ny, Nx);
122     vector<vector<double>> Cezy2 = create2DArray(Ny, Nx);
123     vector<vector<double>> Chx1 = create2DArray(Ny, Nx);
124     vector<vector<double>> Chx2 = create2DArray(Ny, Nx);
125     vector<vector<double>> Chy1 = create2DArray(Ny, Nx);
126     vector<vector<double>> Chy2 = create2DArray(Ny, Nx);
127
128     for (int i = 0; i < Ny; ++i) {
129         for (int j = 0; j < Nx; ++j) {
130             Cezx1[i][j] = (e_0 * e_r[i][j] - 0.5 * dt * s_x[i][j]) / (e_0
                    * e_r[i][j] + 0.5 * dt * s_x[i][j]);
131             Cezx2[i][j] = dt / (dx * (e_0 * e_r[i][j] + 0.5 * dt * s_x[i
                    ][j]));
132             Cezy1[i][j] = (e_0 * e_r[i][j] - 0.5 * dt * s_y[i][j]) / (e_0
                    * e_r[i][j] + 0.5 * dt * s_y[i][j]);
133             Cezy2[i][j] = dt / (dy * (e_0 * e_r[i][j] + 0.5 * dt * s_y[i
                    ][j]));
134             Chx1[i][j] = (mu - 0.5 * dt * ss_x[i][j]) / (mu + 0.5 * dt *
                    ss_x[i][j]);
135             Chx2[i][j] = dt / (dy * (mu + 0.5 * dt * ss_x[i][j]));
136             Chy1[i][j] = (mu - 0.5 * dt * ss_y[i][j]) / (mu + 0.5 * dt *
                    ss_y[i][j]);
137             Chy2[i][j] = dt / (dx * (mu + 0.5 * dt * ss_y[i][j]));
138         }
```

```
139     }
140
141     return {Cezx1, Cezx2, Cezy1, Cezy2, Chx1, Chx2, Chy1, Chy2, e_r};
142 }
143
144 int main() {
145     // Ruimtelijke stap van 2 cm
146     double dx = DX;
147     double dy = DY;
148
149     // Fysische constanten
150     double c = C;
151     double mu = MU;
152     double e_0 = E_0;
153
154     // Lees de ruimte uit het CSV-bestand
155     string filename = "python_room.csv";
156     vector<vector<double>> e_r = readCSV(filename);
157
158     // Tijdstap (2D CFL-voorwaarde)
159     double dt = 1.0 / (c * sqrt((1.0 / (dx * dx)) + (1.0 / (dy * dy))));
160     int Nt = NT;
161
162     // Initialiseer achtergrondvariabelen
163     auto backgroundVars = setBackgroundVariables_I(NX, NY, e_r, e_0, mu,
        dt, dx, dy);
164     vector<vector<double>> Cezx1 = get<0>(backgroundVars);
165     vector<vector<double>> Cezx2 = get<1>(backgroundVars);
166     vector<vector<double>> Cezy1 = get<2>(backgroundVars);
167     vector<vector<double>> Cezy2 = get<3>(backgroundVars);
168     vector<vector<double>> Chx1 = get<4>(backgroundVars);
169     vector<vector<double>> Chx2 = get<5>(backgroundVars);
170     vector<vector<double>> Chy1 = get<6>(backgroundVars);
171     vector<vector<double>> Chy2 = get<7>(backgroundVars);
172     vector<vector<double>> e_r_new = get<8>(backgroundVars);
173
174     // Veldvariabel toewijzing
175     vector<vector<double>> Hx = create2DArray(NX, NY);  // Magnetisch
        veld x-component
176     vector<vector<double>> Hy = create2DArray(NX, NY);  // Magnetisch
        veld y-component
177     vector<vector<double>> Ez = create2DArray(NX, NY);  // Elektrisch
        veld z-component
178     vector<vector<double>> Ezx = create2DArray(NX, NY); // Elektrisch
        veld z-component in de x-richting
179     vector<vector<double>> Ezy = create2DArray(NX, NY); // Elektrisch
        veld z-component in de y-richting
180
181     // Bron definitie
182     double sigma_t = 10 * dt;  // Breedte van de puls
183     double t0 = 3 * sigma_t;   // Centrum van de puls
184     double wavel = 0.15;       // Golflengte [m]
185     double N_lamb = c / wavel; // Frequentie [Hz]
```

```
186
187     // Sinusoidal Source Function
188     auto sinusoid = [N_lamb, dt, t0](double t) {
189         return sin(2 * M_PI * N_lamb * (t - t0) * dt);
190     };
191
192     // Stoorlocatie
193     int x_disturbance = 50;
194     int y_disturbance = 50;
195
196     // Open het bestand voor het schrijven van alle frames
197     ofstream file("EZ_werkend_met_timing.csv");
198
199     // Variables for timing
200     double total_time = 0.0;
201
202     // Itereer van t=0 tot t=Nt-1
203     for (int t = 0; t < Nt; ++t) {
204         // Start timing
205         auto start = high_resolution_clock::now();
206
207         // Solve dtEzx = dxHy
208         for (int iX = 1; iX < NX; ++iX) {
209             for (int iY = 0; iY < NY; ++iY) {
210                 Ezx[iY][iX] = Cezx1[iY][iX] * Ezx[iY][iX] + Cezx2[iY][iX]
                        * (Hy[iY][iX] - Hy[iY][iX - 1]);
211             }
212         }
213
214         // Solve dtEzy = - dyHx
215         for (int iX = 0; iX < NX; ++iX) {
216             for (int iY = 1; iY < NY; ++iY) {
217                 Ezy[iY][iX] = Cezy1[iY][iX] * Ezy[iY][iX] - Cezy2[iY][iX]
                        * (Hx[iY][iX] - Hx[iY - 1][iX]);
218             }
219         }
220
221         // Recouple Ez
222         for (int iX = 0; iX < NX; ++iX) {
223             for (int iY = 0; iY < NY; ++iY) {
224                 Ez[iY][iX] = Ezx[iY][iX] + Ezy[iY][iX];
225             }
226         }
227
228         // Add source to the electric field
229         Ez[x_disturbance][y_disturbance] += sinusoid(t);
230
231         // Solve dtHx = -dyEz
232         for (int iX = 0; iX < NX; ++iX) {
233             for (int iY = 0; iY < NY - 1; ++iY) {
234                 Hx[iY][iX] = Chx1[iY][iX] * Hx[iY][iX] - Chx2[iY][iX] * (
                        Ez[iY + 1][iX] - Ez[iY][iX]);
235             }
```

```
236         }
237
238         // Solve dtHy = dxEz
239         for (int iX = 0; iX < NX - 1; ++iX) {
240             for (int iY = 0; iY < NY; ++iY) {
241                 Hy[iY][iX] = Chy1[iY][iX] * Hy[iY][iX] + Chy2[iY][iX] * (
                        Ez[iY][iX + 1] - Ez[iY][iX]);
242             }
243         }
244
245         // Stop timing
246         auto stop = high_resolution_clock::now();
247         duration<double> elapsed = stop - start;
248         total_time += elapsed.count();
249
250         // Append the E matrix to the CSV file for each frame
251         appendMatrixToCSV(file, Ez, t);
252     }
253
254     // Calculate and print the average time per iteration
255     double average_time = total_time / Nt;
256     cout << "Average time per iteration: " << average_time << " seconds."
            << endl;
257
258     file.close();
259
260     return 0;
261 }
```

8.3 Cuda V1

```
#include <iostream>
#include <cmath>
#include <vector>
#include <fstream>
#include <sstream>
#include <tuple>
#include <algorithm> // For std::transform
#include <cuda_runtime.h> // Include CUDA runtime header

using namespace std;

// Define constants with #define
#define NX 1000
#define NY 1000
#define DX 0.02
#define DY 0.02
#define C 3e8
#define MU (4.0 * M_PI * 1.0e-7)
#define E_0 (1.0 / (C * C * MU))
#define NT 500

// Function to create a 2D matrix of doubles with given dimensions and initial value
vector<vector<double>> create2DArray(int Nx, int Ny, double initial_value = 0.0) {
    return vector<vector<double>>(Nx, vector<double>(Ny, initial_value));
}

// Function to read a CSV file and load the data into a matrix
vector<vector<double>> readCSV(const string& filename) {
    vector<vector<double>> matrix;
    ifstream file(filename);

    if (!file.is_open()) {
        cerr << "Fout bij het openen van bestand: " << filename << endl;
        return matrix;
    }

    string line;
    while (getline(file, line)) {
        stringstream ss(line);
        string value;
        vector<double> row;

        while (getline(ss, value, ',')) {
            row.push_back(stod(value));
        }
        matrix.push_back(row);
    }

    file.close();
```

```
50      return matrix;
51  }
52
53  // Function to write a matrix to a CSV file
54  void appendMatrixToCSV(ofstream &file, const vector<vector<double>>&
        matrix, int t) {
55      if (!file.is_open()) {
56          cerr << "Fout bij het openen van bestand voor append: " << endl;
57          return;
58      }
59
60      // Write a header or delimiter for each frame
61      file << "Frame " << t << endl;
62
63      for (const auto& row : matrix) {
64          for (size_t i = 0; i < row.size(); ++i) {
65              file << row[i];
66              if (i < row.size() - 1) {
67                  file << ",";
68              }
69          }
70          file << endl;
71      }
72
73      // Optionally, write a blank line to separate frames
74      file << endl;
75  }
76
77  // Function to set background variables
78  tuple<vector<vector<double>>, vector<vector<double>>, vector<vector<
        double>>, vector<vector<double>>, vector<vector<double>>, vector<
        vector<double>>, vector<vector<double>>, vector<vector<double>>,
        vector<vector<double>>> setBackgroundVariables_I(int Nx, int Ny,
        vector<vector<double>> e_r, double e_0, double mu, double dt, double
        dx, double dy) {
79      int dLayers = 10;  // Number of layers
80      int order = 3;  // Order of polynomial of conductivity values
81
82      vector<vector<double>> s_x = create2DArray(Ny, Nx);
83      vector<vector<double>> s_y = create2DArray(Ny, Nx);  // Electric
            conductivities
84
85      for (int iX = 0; iX < Nx; ++iX) {
86          for (int j = 0; j < dLayers; ++j) {
87              s_x[iX][j] = pow(dLayers - 1 - j, order) / pow(dLayers - 1,
                    order);
88              s_x[iX][Nx - dLayers + j] = pow(j, order) / pow(dLayers - 1,
                    order);
89          }
90      }
91
92      for (int iY = 0; iY < Ny; ++iY) {
93          for (int j = 0; j < dLayers; ++j) {
```

```
94              s_y[j][iY] = pow(dLayers - 1 - j, order) / pow(dLayers - 1,
                    order);
95              s_y[Ny - dLayers + j][iY] = pow(j, order) / pow(dLayers - 1,
                    order);
96          }
97      }
98
99      vector<vector<double>> ss_x = s_x;
100     vector<vector<double>> ss_y = s_y;
101
102     for (int i = 0; i < Ny; ++i) {
103         for (int j = 0; j < Nx; ++j) {
104             ss_x[i][j] *= mu / e_0;
105             ss_y[i][j] *= mu / e_0;
106         }
107     }
108
109     for (int i = dLayers - 1; i < Ny - dLayers; ++i) {
110         for (int j = dLayers - 1; j < Nx - dLayers; ++j) {
111             if (e_r[i][j] != 1) {
112                 s_x[i][j] = e_r[i][j] / 40.0;
113                 s_y[i][j] = e_r[i][j] / 40.0;
114             }
115         }
116     }
117
118     vector<vector<double>> Cezx1 = create2DArray(Ny, Nx);
119     vector<vector<double>> Cezx2 = create2DArray(Ny, Nx);
120     vector<vector<double>> Cezy1 = create2DArray(Ny, Nx);
121     vector<vector<double>> Cezy2 = create2DArray(Ny, Nx);
122     vector<vector<double>> Chx1 = create2DArray(Ny, Nx);
123     vector<vector<double>> Chx2 = create2DArray(Ny, Nx);
124     vector<vector<double>> Chy1 = create2DArray(Ny, Nx);
125     vector<vector<double>> Chy2 = create2DArray(Ny, Nx);
126
127     for (int i = 0; i < Ny; ++i) {
128         for (int j = 0; j < Nx; ++j) {
129             Cezx1[i][j] = (e_0 * e_r[i][j] - 0.5 * dt * s_x[i][j]) / (e_0
                    * e_r[i][j] + 0.5 * dt * s_x[i][j]);
130             Cezx2[i][j] = dt / (dx * (e_0 * e_r[i][j] + 0.5 * dt * s_x[i
                    ][j]));
131
132             Cezy1[i][j] = (e_0 * e_r[i][j] - 0.5 * dt * s_y[i][j]) / (e_0
                    * e_r[i][j] + 0.5 * dt * s_y[i][j]);
133             Cezy2[i][j] = dt / (dy * (e_0 * e_r[i][j] + 0.5 * dt * s_y[i
                    ][j]));
134
135             Chx1[i][j] = (mu - 0.5 * dt * ss_x[i][j]) / (mu + 0.5 * dt *
                    ss_x[i][j]);
136             Chx2[i][j] = dt / (dy * (mu + 0.5 * dt * ss_x[i][j]));
137
138             Chy1[i][j] = (mu - 0.5 * dt * ss_y[i][j]) / (mu + 0.5 * dt *
                    ss_y[i][j]);
```

```
139             Chy2[i][j] = dt / (dx * (mu + 0.5 * dt * ss_y[i][j]));
140         }
141     }
142
143     return {Cezx1, Cezx2, Cezy1, Cezy2, Chx1, Chx2, Chy1, Chy2, e_r};
144 }
145
146 // Copy 2D vector to 1D array
147 void copy2DVectorTo1DArray(double* device_array, const vector<vector<
    double>>& host_matrix, int Nx, int Ny) {
148     vector<double> host_array(Nx * Ny);
149     for (int i = 0; i < Ny; ++i) {
150         for (int j = 0; j < Nx; ++j) {
151             host_array[i * Nx + j] = host_matrix[i][j];
152         }
153     }
154     cudaMemcpy(device_array, host_array.data(), Nx * Ny * sizeof(double),
         cudaMemcpyHostToDevice);
155 }
156
157 // Copy 1D array to 2D vector
158 void copy1DArrayTo2DVector(vector<vector<double>>& host_matrix, double*
    device_array, int Nx, int Ny) {
159     vector<double> host_array(Nx * Ny);
160     cudaMemcpy(host_array.data(), device_array, Nx * Ny * sizeof(double),
         cudaMemcpyDeviceToHost);
161     for (int i = 0; i < Ny; ++i) {
162         for (int j = 0; j < Nx; ++j) {
163             host_matrix[i][j] = host_array[i * Nx + j];
164         }
165     }
166 }
167
168 // CUDA kernel to update Ezx
169 __global__ void updateEzx(double* Ezx, double* Hy, double* Cezx1, double*
     Cezx2, int Nx, int Ny) {
170     int iX = blockIdx.x * blockDim.x + threadIdx.x;
171     int iY = blockIdx.y * blockDim.y + threadIdx.y;
172
173     if (iX > 0 && iX < Nx && iY < Ny) {
174         int idx = iY * Nx + iX;
175         int idx_left = iY * Nx + (iX - 1);
176         Ezx[idx] = Cezx1[idx] * Ezx[idx] + Cezx2[idx] * (Hy[idx] - Hy[
            idx_left]);
177     }
178 }
179
180 // CUDA kernel to update Ezy
181 __global__ void updateEzy(double* Ezy, double* Hx, double* Cezy1, double*
     Cezy2, int Nx, int Ny) {
182     int iX = blockIdx.x * blockDim.x + threadIdx.x;
183     int iY = blockIdx.y * blockDim.y + threadIdx.y;
184
```

```
185     if (iY > 0 && iX < Nx && iY < Ny) {
186         int idx = iY * Nx + iX;
187         int idx_below = (iY - 1) * Nx + iX;
188         Ezy[idx] = Cezy1[idx] * Ezy[idx] - Cezy2[idx] * (Hx[idx] - Hx[
                idx_below]);
189     }
190 }
191 
192 // CUDA kernel to recouple Ez
193 __global__ void recoupleEz(double* Ez, double* Ezx, double* Ezy, int Nx,
        int Ny) {
194     int iX = blockIdx.x * blockDim.x + threadIdx.x;
195     int iY = blockIdx.y * blockDim.y + threadIdx.y;
196 
197     if (iX < Nx && iY < Ny) {
198         int idx = iY * Nx + iX;
199         Ez[idx] = Ezx[idx] + Ezy[idx];
200     }
201 }
202 
203 // CUDA kernel to add source to Ez
204 __global__ void addSource(double* Ez, double source_value, int
        x_disturbance, int y_disturbance, int Nx) {
205     int idx = y_disturbance * Nx + x_disturbance;
206     Ez[idx] += source_value;
207 }
208 
209 // CUDA kernel to update Hx
210 __global__ void updateHx(double* Hx, double* Ez, double* Chx1, double*
        Chx2, int Nx, int Ny) {
211     int iX = blockIdx.x * blockDim.x + threadIdx.x;
212     int iY = blockIdx.y * blockDim.y + threadIdx.y;
213 
214     if (iX < Nx && iY < Ny - 1) {
215         int idx = iY * Nx + iX;
216         int idx_above = (iY + 1) * Nx + iX;
217         Hx[idx] = Chx1[idx] * Hx[idx] - Chx2[idx] * (Ez[idx_above] - Ez[
                idx]);
218     }
219 }
220 
221 // CUDA kernel to update Hy
222 __global__ void updateHy(double* Hy, double* Ez, double* Chy1, double*
        Chy2, int Nx, int Ny) {
223     int iX = blockIdx.x * blockDim.x + threadIdx.x;
224     int iY = blockIdx.y * blockDim.y + threadIdx.y;
225 
226     if (iX < Nx - 1 && iY < Ny) {
227         int idx = iY * Nx + iX;
228         int idx_right = iY * Nx + (iX + 1);
229         Hy[idx] = Chy1[idx] * Hy[idx] + Chy2[idx] * (Ez[idx_right] - Ez[
                idx]);
230     }
```

```
231 }
232
233 int main() {
234     // Ruimtelijke stap van 2 cm
235     double dx = DX;
236     double dy = DY;
237
238     // Fysische constanten
239     double c = C;
240     double mu = MU;
241     double e_0 = E_0;
242
243     // Lees de ruimte uit het CSV-bestand
244     string filename = "python_room_1000x1000.csv";
245     vector<vector<double>> e_r = readCSV(filename);
246
247     // Tijdstap (2D CFL-voorwaarde)
248     double dt = 1.0 / (c * sqrt((1.0 / (dx * dx)) + (1.0 / (dy * dy))));
249     int Nt = NT;
250
251     // Initialiseer achtergrondvariabelen
252     auto backgroundVars = setBackgroundVariables_I(NX, NY, e_r, e_0, mu,
        dt, dx, dy);
253     vector<vector<double>> Cezx1 = get<0>(backgroundVars);
254     vector<vector<double>> Cezx2 = get<1>(backgroundVars);
255     vector<vector<double>> Cezy1 = get<2>(backgroundVars);
256     vector<vector<double>> Cezy2 = get<3>(backgroundVars);
257     vector<vector<double>> Chx1 = get<4>(backgroundVars);
258     vector<vector<double>> Chx2 = get<5>(backgroundVars);
259     vector<vector<double>> Chy1 = get<6>(backgroundVars);
260     vector<vector<double>> Chy2 = get<7>(backgroundVars);
261     vector<vector<double>> e_r_new = get<8>(backgroundVars);
262
263     // Veldvariabel toewijzing
264     vector<vector<double>> Hx = create2DArray(NX, NY);  // Magnetisch
        veld x-component
265     vector<vector<double>> Hy = create2DArray(NX, NY);  // Magnetisch
        veld y-component
266     vector<vector<double>> Ez = create2DArray(NX, NY);  // Elektrisch
        veld z-component
267     vector<vector<double>> Ezx = create2DArray(NX, NY); // Elektrisch
        veld z-component in de x-richting
268     vector<vector<double>> Ezy = create2DArray(NX, NY); // Elektrisch
        veld z-component in de y-richting
269
270     // Bron definitie
271     double sigma_t = 10 * dt;  // Breedte van de puls
272     double t0 = 3 * sigma_t;   // Centrum van de puls
273     double wavel = 0.15;       // Golflengte [m]
274     double N_lamb = c / wavel; // Frequentie [Hz]
275
276     // Sinusoidal Source Function
277     auto sinusoid = [N_lamb, dt, t0](double t) {
```

```
278         return sin(2 * M_PI * N_lamb * (t - t0) * dt);
279     };
280 
281     // Stoorlocatie
282     int x_disturbance = 50;
283     int y_disturbance = 50;
284 
285     // Allocate device memory
286     double *d_Ezx, *d_Hy, *d_Cezx1, *d_Cezx2, *d_Ezy, *d_Hx, *d_Cezy1, *
            d_Cezy2, *d_Ez, *d_Hx1, *d_Chx1, *d_Chx2, *d_Hy1, *d_Chy1, *d_Chy2
            ;
287     size_t size = NX * NY * sizeof(double);
288 
289     cudaMalloc(&d_Ezx, size);
290     cudaMalloc(&d_Hy, size);
291     cudaMalloc(&d_Cezx1, size);
292     cudaMalloc(&d_Cezx2, size);
293     cudaMalloc(&d_Ezy, size);
294     cudaMalloc(&d_Hx, size);
295     cudaMalloc(&d_Cezy1, size);
296     cudaMalloc(&d_Cezy2, size);
297     cudaMalloc(&d_Ez, size);
298     cudaMalloc(&d_Hx1, size);
299     cudaMalloc(&d_Chx1, size);
300     cudaMalloc(&d_Chx2, size);
301     cudaMalloc(&d_Hy1, size);
302     cudaMalloc(&d_Chy1, size);
303     cudaMalloc(&d_Chy2, size);
304 
305     // Copy data to device
306     copy2DVectorTo1DArray(d_Ezx, Ezx, NX, NY);
307     copy2DVectorTo1DArray(d_Hy, Hy, NX, NY);
308     copy2DVectorTo1DArray(d_Cezx1, Cezx1, NX, NY);
309     copy2DVectorTo1DArray(d_Cezx2, Cezx2, NX, NY);
310     copy2DVectorTo1DArray(d_Ezy, Ezy, NX, NY);
311     copy2DVectorTo1DArray(d_Hx, Hx, NX, NY);
312     copy2DVectorTo1DArray(d_Cezy1, Cezy1, NX, NY);
313     copy2DVectorTo1DArray(d_Cezy2, Cezy2, NX, NY);
314     copy2DVectorTo1DArray(d_Ez, Ez, NX, NY);
315     copy2DVectorTo1DArray(d_Hx1, Hx, NX, NY);
316     copy2DVectorTo1DArray(d_Chx1, Chx1, NX, NY);
317     copy2DVectorTo1DArray(d_Chx2, Chx2, NX, NY);
318     copy2DVectorTo1DArray(d_Hy1, Hy, NX, NY);
319     copy2DVectorTo1DArray(d_Chy1, Chy1, NX, NY);
320     copy2DVectorTo1DArray(d_Chy2, Chy2, NX, NY);
321 
322     // Define grid and block dimensions
323     dim3 blockDim(16, 16);
324     dim3 gridDim((NX + blockDim.x - 1) / blockDim.x, (NY + blockDim.y -
            1) / blockDim.y);
325 
326     // Open the file for writing all frames
327     ofstream file("v1_werkend_1000.csv");
```

```
328
329     cudaEvent_t start, stop;
330     cudaEventCreate(&start);
331     cudaEventCreate(&stop);
332     float millis;
333     double total_time = 0.0;
334     // Iterate from t=0 to t=Nt-1
335     for (int t = 0; t < Nt; ++t) {
336
337         cudaEventRecord(start);
338         // Update Ezx on GPU
339         updateEzx<<<gridDim, blockDim>>>(d_Ezx, d_Hy, d_Cezx1, d_Cezx2,
                NX, NY);
340
341         // Update Ezy on GPU
342         updateEzy<<<gridDim, blockDim>>>(d_Ezy, d_Hx, d_Cezy1, d_Cezy2,
                NX, NY);
343
344         // Recouple Ez on GPU
345         recoupleEz<<<gridDim, blockDim>>>(d_Ez, d_Ezx, d_Ezy, NX, NY);
346
347         // Add source to the electric field on GPU
348         double source_value = sinusoid(t);
349         addSource<<<1, 1>>>(d_Ez, source_value, x_disturbance,
                y_disturbance, NX);
350
351         // Update Hx on GPU
352         updateHx<<<gridDim, blockDim>>>(d_Hx, d_Ez, d_Chx1, d_Chx2, NX,
                NY);
353
354         // Update Hy on GPU
355         updateHy<<<gridDim, blockDim>>>(d_Hy, d_Ez, d_Chy1, d_Chy2, NX,
                NY);
356         cudaEventRecord(stop);
357         cudaEventSynchronize(stop);
358         cudaEventElapsedTime(&millis, start, stop);
359         total_time += millis;
360         // Copy Ez back to host for writing to file
361         copy1DArrayTo2DVector(Ez, d_Ez, NX, NY);
362
363         // Append the Ez matrix to the CSV file for each frame
364         appendMatrixToCSV(file, Ez, t);
365     }
366
367     // Free device memory
368     cudaFree(d_Ezx);
369     cudaFree(d_Hy);
370     cudaFree(d_Cezx1);
371     cudaFree(d_Cezx2);
372     cudaFree(d_Ezy);
373     cudaFree(d_Hx);
374     cudaFree(d_Cezy1);
375     cudaFree(d_Cezy2);
```

```
376     cudaFree(d_Ez);
377     cudaFree(d_Hx1);
378     cudaFree(d_Chx1);
379     cudaFree(d_Chx2);
380     cudaFree(d_Hy1);
381     cudaFree(d_Chy1);
382     cudaFree(d_Chy2);
383 
384     double average_time = total_time / (Nt*1000);
385     cout << "Average time per iteration: " << average_time << " seconds."
            << endl;
386     // Close the file
387     file.close();
388 
389     return 0;
390 }
```

8.4 Cuda V2

```
// CUDA kernel to update both Ezx and Ezy
__global__ void updateEz(double* Ezx, double* Ezy, double* Hy, double* Hx,
                         double* Cezx1, double* Cezx2,
                         double* Cezy1, double* Cezy2,
                         int Nx, int Ny) {
    int iX = blockIdx.x * blockDim.x + threadIdx.x;
    int iY = blockIdx.y * blockDim.y + threadIdx.y;

    // Check if within bounds
    if (iX < Nx && iY < Ny) {
        int idx = iY * Nx + iX;

        // Update Ezx
        if (iX > 0) {  // Ensure not out of bounds on the left
            int idx_left = iY * Nx + (iX - 1);
            Ezx[idx] = Cezx1[idx] * Ezx[idx] + Cezx2[idx] * (Hy[idx] - Hy[idx_left]);
        }

        // Update Ezy
        if (iY > 0) {  // Ensure not out of bounds below
            int idx_below = (iY - 1) * Nx + iX;
            Ezy[idx] = Cezy1[idx] * Ezy[idx] - Cezy2[idx] * (Hx[idx] - Hx[idx_below]);
        }
    }
}

// CUDA kernel to recouple Ez
__global__ void recoupleEz(double* Ez, double* Ezx, double* Ezy, int Nx, int Ny) {
    int iX = blockIdx.x * blockDim.x + threadIdx.x;
    int iY = blockIdx.y * blockDim.y + threadIdx.y;

    if (iX < Nx && iY < Ny) {
        int idx = iY * Nx + iX;
        Ez[idx] = Ezx[idx] + Ezy[idx];
    }
}

// CUDA kernel to add source to Ez
__global__ void addSource(double* Ez, double source_value, int x_disturbance, int y_disturbance, int Nx) {
    int idx = y_disturbance * Nx + x_disturbance;
    Ez[idx] += source_value;
}

// CUDA kernel to update Hx
__global__ void updateHx(double* Hx, double* Ez, double* Chx1, double* Chx2, int Nx, int Ny) {
```

```
46      int iX = blockIdx.x * blockDim.x + threadIdx.x;
47      int iY = blockIdx.y * blockDim.y + threadIdx.y;
48
49      if (iX < Nx && iY < Ny - 1) {
50          int idx = iY * Nx + iX;
51          int idx_above = (iY + 1) * Nx + iX;
52          Hx[idx] = Chx1[idx] * Hx[idx] - Chx2[idx] * (Ez[idx_above] - Ez[
                idx]);
53      }
54  }
55
56  // CUDA kernel to update Hy
57  __global__ void updateHy(double* Hy, double* Ez, double* Chy1, double*
        Chy2, int Nx, int Ny) {
58      int iX = blockIdx.x * blockDim.x + threadIdx.x;
59      int iY = blockIdx.y * blockDim.y + threadIdx.y;
60
61      if (iX < Nx - 1 && iY < Ny) {
62          int idx = iY * Nx + iX;
63          int idx_right = iY * Nx + (iX + 1);
64          Hy[idx] = Chy1[idx] * Hy[idx] + Chy2[idx] * (Ez[idx_right] - Ez[
                idx]);
65      }
66  }
```

8.5 Cuda V3

```
__global__ void updateEz(double* Ez, double* Ezx, double* Ezy, double* Hy
    , double* Hx,
                                        double* Cezx1, double* Cezx2,
                                        double* Cezy1, double* Cezy2,
                                        int Nx, int Ny) {
    int iX = blockIdx.x * blockDim.x + threadIdx.x;
    int iY = blockIdx.y * blockDim.y + threadIdx.y;

    // Check if within bounds
    if (iX < Nx && iY < Ny) {
        int idx = iY * Nx + iX;

        // Update Ezx
        if (iX > 0) {  // Ensure not out of bounds on the left
            int idx_left = iY * Nx + (iX - 1);
            Ezx[idx] = Cezx1[idx] * Ezx[idx] + Cezx2[idx] * (Hy[idx] - Hy
                [idx_left]);
        }

        // Update Ezy
        if (iY > 0) {  // Ensure not out of bounds below
            int idx_below = (iY - 1) * Nx + iX;
            Ezy[idx] = Cezy1[idx] * Ezy[idx] - Cezy2[idx] * (Hx[idx] - Hx
                [idx_below]);
        }

        // Recouple Ez (sum Ezx and Ezy)
        Ez[idx] = Ezx[idx] + Ezy[idx];
    }
}

// CUDA kernel to add source to Ez
__global__ void addSource(double* Ez, double source_value, int
    x_disturbance, int y_disturbance, int Nx) {
    int idx = y_disturbance * Nx + x_disturbance;
    Ez[idx] += source_value;
}

// CUDA kernel to update Hx
__global__ void updateHx(double* Hx, double* Ez, double* Chx1, double*
    Chx2, int Nx, int Ny) {
    int iX = blockIdx.x * blockDim.x + threadIdx.x;
    int iY = blockIdx.y * blockDim.y + threadIdx.y;

    if (iX < Nx && iY < Ny - 1) {
        int idx = iY * Nx + iX;
        int idx_above = (iY + 1) * Nx + iX;
        Hx[idx] = Chx1[idx] * Hx[idx] - Chx2[idx] * (Ez[idx_above] - Ez[
            idx]);
    }
}
```

```
46
47 // CUDA kernel to update Hy
48 __global__ void updateHy(double* Hy, double* Ez, double* Chy1, double*
       Chy2, int Nx, int Ny) {
49     int iX = blockIdx.x * blockDim.x + threadIdx.x;
50     int iY = blockIdx.y * blockDim.y + threadIdx.y;
51
52     if (iX < Nx - 1 && iY < Ny) {
53         int idx = iY * Nx + iX;
54         int idx_right = iY * Nx + (iX + 1);
55         Hy[idx] = Chy1[idx] * Hy[idx] + Chy2[idx] * (Ez[idx_right] - Ez[
               idx]);
56     }
57 }
```

8.6 Cuda V4

```
__global__ void updateEz(double* Ez, double* Ezx, double* Ezy, double* Hy, double* Hx,
                                    double* Cezx1, double* Cezx2,
                                    double* Cezy1, double* Cezy2,
                                    int Nx, int Ny) {
    int iX = blockIdx.x * blockDim.x + threadIdx.x;
    int iY = blockIdx.y * blockDim.y + threadIdx.y;

    // Check if within bounds
    if (iX < Nx && iY < Ny) {
        int idx = iY * Nx + iX;

        // Update Ezx
        if (iX > 0) {  // Ensure not out of bounds on the left
            int idx_left = iY * Nx + (iX - 1);
            Ezx[idx] = Cezx1[idx] * Ezx[idx] + Cezx2[idx] * (Hy[idx] - Hy[idx_left]);
        }

        // Update Ezy
        if (iY > 0) {  // Ensure not out of bounds below
            int idx_below = (iY - 1) * Nx + iX;
            Ezy[idx] = Cezy1[idx] * Ezy[idx] - Cezy2[idx] * (Hx[idx] - Hx[idx_below]);
        }

        // Recouple Ez (sum Ezx and Ezy)
        Ez[idx] = Ezx[idx] + Ezy[idx];
    }
}

// CUDA kernel to add source to Ez
__global__ void addSource(double* Ez, double source_value, int x_disturbance, int y_disturbance, int Nx) {
    int idx = y_disturbance * Nx + x_disturbance;
    Ez[idx] += source_value;
}

__global__ void updateH(double* Hx, double* Hy, double* Ez, double* Chx1, double* Chx2, double* Chy1, double* Chy2,
                        int Nx, int Ny) {
    int iX = blockIdx.x * blockDim.x + threadIdx.x;
    int iY = blockIdx.y * blockDim.y + threadIdx.y;

    // Update Hx
    if (iX < Nx && iY < Ny - 1) {
        int idx = iY * Nx + iX;
        int idx_above = (iY + 1) * Nx + iX;
        Hx[idx] = Chx1[idx] * Hx[idx] - Chx2[idx] * (Ez[idx_above] - Ez[idx]);
    }
```

```
46
47      // Update Hy
48      if (iX < Nx - 1 && iY < Ny) {
49          int idx = iY * Nx + iX;
50          int idx_right = iY * Nx + (iX + 1);
51          Hy[idx] = Chy1[idx] * Hy[idx] + Chy2[idx] * (Ez[idx_right] - Ez[
                idx]);
52      }
53  }
```

8.7 Cuda V5

```
__global__ void updateEz(double* Ez, double* Ezx, double* Ezy, double* Hy
    , double* Hx,
                         double* Cezx1, double* Cezx2,
                         double* Cezy1, double* Cezy2,
                         int Nx, int Ny) {
    int iX = blockIdx.x * blockDim.x + threadIdx.x;
    int iY = blockIdx.y * blockDim.y + threadIdx.y;
    int idx = iY * Nx + iX;

    // Check if within bounds
    if (iX < Nx && iY < Ny) {
        // Using shared memory for coefficients and fields
        __shared__ double s_Cezx1[16][16];
        __shared__ double s_Cezx2[16][16];
        __shared__ double s_Cezy1[16][16];
        __shared__ double s_Cezy2[16][16];
        __shared__ double s_Hy[16][16];
        __shared__ double s_Hx[16][16];

        s_Cezx1[threadIdx.y][threadIdx.x] = Cezx1[idx];
        s_Cezx2[threadIdx.y][threadIdx.x] = Cezx2[idx];
        s_Cezy1[threadIdx.y][threadIdx.x] = Cezy1[idx];
        s_Cezy2[threadIdx.y][threadIdx.x] = Cezy2[idx];
        s_Hy[threadIdx.y][threadIdx.x] = Hy[idx];
        s_Hx[threadIdx.y][threadIdx.x] = Hx[idx];

        __syncthreads();

        // Update Ezx
        if (iX > 0) {  // Ensure not out of bounds on the left
            int idx_left = iY * Nx + (iX - 1);
            Ezx[idx] = s_Cezx1[threadIdx.y][threadIdx.x] * Ezx[idx] +
                       s_Cezx2[threadIdx.y][threadIdx.x] * (s_Hy[
                           threadIdx.y][threadIdx.x] - Hy[idx_left]);
        }

        // Update Ezy
        if (iY > 0) {  // Ensure not out of bounds below
            int idx_below = (iY - 1) * Nx + iX;
            Ezy[idx] = s_Cezy1[threadIdx.y][threadIdx.x] * Ezy[idx] -
                       s_Cezy2[threadIdx.y][threadIdx.x] * (s_Hx[
                           threadIdx.y][threadIdx.x] - Hx[idx_below]);
        }

        __syncthreads();

        // Recouple Ez (sum Ezx and Ezy)
        Ez[idx] = Ezx[idx] + Ezy[idx];
    }
}

```

```
49 // CUDA kernel to add source to Ez
50 __global__ void addSource(double* Ez, double source_value, int
       x_disturbance, int y_disturbance, int Nx) {
51     int idx = y_disturbance * Nx + x_disturbance;
52     Ez[idx] += source_value;
53 }
54
55 __global__ void updateH(double* Hx, double* Hy, double* Ez, double* Chx1,
        double* Chx2, double* Chy1, double* Chy2,
56                             int Nx, int Ny) {
57     int iX = blockIdx.x * blockDim.x + threadIdx.x;
58     int iY = blockIdx.y * blockDim.y + threadIdx.y;
59
60     // Update Hx
61     if (iX < Nx && iY < Ny - 1) {
62         int idx = iY * Nx + iX;
63         int idx_above = (iY + 1) * Nx + iX;
64         Hx[idx] = Chx1[idx] * Hx[idx] - Chx2[idx] * (Ez[idx_above] - Ez[
               idx]);
65     }
66
67     // Update Hy
68     if (iX < Nx - 1 && iY < Ny) {
69         int idx = iY * Nx + iX;
70         int idx_right = iY * Nx + (iX + 1);
71         Hy[idx] = Chy1[idx] * Hy[idx] + Chy2[idx] * (Ez[idx_right] - Ez[
               idx]);
72     }
73 }
```

8.8 Cuda V6

```
__global__ void updateEz(double* Ez, double* Ezx, double* Ezy, double* Hy
    , double* Hx,
                         double* Cezx1, double* Cezx2,
                         double* Cezy1, double* Cezy2,
                         int Nx, int Ny) {
    int iX = blockIdx.x * blockDim.x + threadIdx.x;
    int iY = blockIdx.y * blockDim.y + threadIdx.y;
    int idx = iY * Nx + iX;

    // Check if within bounds
    if (iX < Nx && iY < Ny) {
        // Using shared memory for coefficients and fields
        __shared__ double s_Cezx1[16][16];
        __shared__ double s_Cezx2[16][16];
        __shared__ double s_Cezy1[16][16];
        __shared__ double s_Cezy2[16][16];
        __shared__ double s_Hy[16][16];
        __shared__ double s_Hx[16][16];

        s_Cezx1[threadIdx.y][threadIdx.x] = Cezx1[idx];
        s_Cezx2[threadIdx.y][threadIdx.x] = Cezx2[idx];
        s_Cezy1[threadIdx.y][threadIdx.x] = Cezy1[idx];
        s_Cezy2[threadIdx.y][threadIdx.x] = Cezy2[idx];
        s_Hy[threadIdx.y][threadIdx.x] = Hy[idx];
        s_Hx[threadIdx.y][threadIdx.x] = Hx[idx];

        __syncthreads();

        // Update Ezx
        if (iX > 0) {  // Ensure not out of bounds on the left
            int idx_left = iY * Nx + (iX - 1);
            Ezx[idx] = s_Cezx1[threadIdx.y][threadIdx.x] * Ezx[idx] +
                       s_Cezx2[threadIdx.y][threadIdx.x] * (s_Hy[
                           threadIdx.y][threadIdx.x] - Hy[idx_left]);
        }

        // Update Ezy
        if (iY > 0) {  // Ensure not out of bounds below
            int idx_below = (iY - 1) * Nx + iX;
            Ezy[idx] = s_Cezy1[threadIdx.y][threadIdx.x] * Ezy[idx] -
                       s_Cezy2[threadIdx.y][threadIdx.x] * (s_Hx[
                           threadIdx.y][threadIdx.x] - Hx[idx_below]);
        }

        __syncthreads();

        // Recouple Ez (sum Ezx and Ezy)
        Ez[idx] = Ezx[idx] + Ezy[idx];
    }
}

```

```
49 // CUDA kernel to add source to Ez
50 __global__ void addSource(double* Ez, double source_value, int
       x_disturbance, int y_disturbance, int Nx) {
51     int idx = y_disturbance * Nx + x_disturbance;
52     Ez[idx] += source_value;
53 }
54
55 __global__ void updateH(double* Hx, double* Hy, double* Ez, double* Chx1,
        double* Chx2, double* Chy1, double* Chy2,
56                         int Nx, int Ny) {
57     int iX = blockIdx.x * blockDim.x + threadIdx.x;
58     int iY = blockIdx.y * blockDim.y + threadIdx.y;
59     int idx = iY * Nx + iX;
60
61     // Define shared memory
62     __shared__ double s_Ez[16][16];
63     __shared__ double s_Chx1[16][16];
64     __shared__ double s_Chx2[16][16];
65     __shared__ double s_Chy1[16][16];
66     __shared__ double s_Chy2[16][16];
67
68     // Load data into shared memory
69     if (iX < Nx && iY < Ny) {
70         s_Ez[threadIdx.y][threadIdx.x] = Ez[idx];
71         s_Chx1[threadIdx.y][threadIdx.x] = Chx1[idx];
72         s_Chx2[threadIdx.y][threadIdx.x] = Chx2[idx];
73         s_Chy1[threadIdx.y][threadIdx.x] = Chy1[idx];
74         s_Chy2[threadIdx.y][threadIdx.x] = Chy2[idx];
75     }
76
77     __syncthreads();
78
79     // Update Hx
80     if (iX < Nx && iY < Ny - 1) {
81         int idx_above = (iY + 1) * Nx + iX;
82         Hx[idx] = s_Chx1[threadIdx.y][threadIdx.x] * Hx[idx] - s_Chx2[
               threadIdx.y][threadIdx.x] * (Ez[idx_above] - s_Ez[threadIdx.y
               ][threadIdx.x]);
83     }
84
85     // Update Hy
86     if (iX < Nx - 1 && iY < Ny) {
87         int idx_right = iY * Nx + (iX + 1);
88         Hy[idx] = s_Chy1[threadIdx.y][threadIdx.x] * Hy[idx] + s_Chy2[
               threadIdx.y][threadIdx.x] * (Ez[idx_right] - s_Ez[threadIdx.y
               ][threadIdx.x]);
89     }
90 }
```

8.9 CSV vergelijker

```python
import csv

def read_csv(file_path):
    with open(file_path, mode='r') as file:
        reader = csv.reader(file)
        return [row for row in reader]

def compare_csv(file1, file2):
    data1 = read_csv(file1)
    data2 = read_csv(file2)

    if data1 == data2:
        print("The␣CSV␣files␣are␣the␣same.")
    else:
        print("The␣CSV␣files␣are␣different.")

# File paths
file1 = 'v6_hy_opt.csv'
file2 = 'v1_werkend.csv'

# Compare the CSV files
compare_csv(file1, file2)
```

8.10 CSV visualiseren

```python
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap
from IPython import display

# Define the custom colormap
cMap = [(0, "Yellow"), (1, "Black")]
custom_cmap = LinearSegmentedColormap.from_list("custom", cMap)

# Parameters
Nx, Ny = 100, 100  # Adjust grid size if known

# Load the file while skipping frame headers
frames = []
csv_path = 'Ez_all_frames.csv'
with open(csv_path, 'r') as file:
    frame_data = []
    for line in file:
        if line.startswith("Frame"):  # Skip frame headers
            if frame_data:  # Save completed frame
                frames.append(np.array(frame_data))
                frame_data = []
        else:
            # Skip empty lines or lines with invalid data
            if line.strip():  # Only process non-empty lines
                try:
                    frame_data.append(list(map(float, line.strip().split(
                        ','))))
                except ValueError:
                    print(f"Skipping␣invalid␣line:␣{line.strip()}")
    if frame_data:  # Add the last frame
        frames.append(np.array(frame_data))

# Convert list of frames to a 3D numpy array
frames = np.array(frames)

# Visualize each frame
Nt = frames.shape[0]  # Total time steps
for t in range(Nt):
    Ez = frames[t]
    intensity = np.abs(Ez) ** 2  # Calculate intensity
    plt.imshow((Ez - 1), cmap=custom_cmap, alpha=1)  # Background map
    plt.imshow(intensity, cmap='viridis', interpolation='nearest', origin
        ='lower', alpha=0.5)
    plt.colorbar(label='Intensity␣of␣$E_z$')
    plt.title(f'Intensity␣of␣$E_z$␣at␣Time␣Step:␣{t}')
    plt.xlabel('X␣Position')
    plt.ylabel('Y␣Position')
    plt.pause(0.1)  # Pause for a brief moment to view the plot
    display.clear_output(wait=True)
```